# SENG 300 — Iteration 3 Summary of Major Changes

**Project:** Online Multiplayer Game Platform (Java / JavaFX)
**Team:** Bogo Solutions
**Document:** Aligns design documentation from Iteration 1 with the **as-built** codebase at iteration end.

---

## 1. Executive summary

The deliverable is a **platform-first** multiplayer hub: authentication, lobbies, matchmaking, chat, profiles, stats, and leaderboards, with **two sample games** — **Tic-Tac-Toe** and **Connect Four** — exercising shared turn handling and validation. Persistence and remote hosting are **stubbed or file-backed** where the course expects future database/cloud integration. This summary records **deliberate design adjustments** relative to Iteration 1 UML and structure docs under `docs/`.

---

## 2. Integration outcomes (end-to-end)

| Flow | As implemented |
|---|---|
| Auth | UI controllers → `ClientCore` → TCP to `PlatformCore`; `UserStore` / hashing (`Helper`) for accounts. |
| Lobby / rooms | `ServerLobby`, lobby controllers, `platformChannel` for connected clients. |
| Matchmaking | `MatchMakingService` (queues by game type, score/rank gap heuristics). |
| Gameplay | Moves as `PlayerAction` subclasses over sockets; `TicTacToeSession` / `ConnectFour` game types in `game.core` + `platform.session`; UI refresh via `refreshFromGameStateUpdate(...)`. |
| Stats / history | `PlayerStatsStore`, `GameHistoryStore`, `ParticipantStore`, etc., JSON via Gson + `Store` base. |
| Stubs | `platform.stubs.Network` (queue pair), `CredentialsDatabase` (credential boundary; course DB still external). |

---

## 3. Iteration 1 baseline vocabulary (planned)

The following **named elements** appear in Iteration 1 artifacts (`docs/Iteration 1 Submission/Full Architecture.md`, `docs/02 structure/*.md`, sequence docs):

- **UI (paper design):** `WelcomePage`, `AuthenticationScreen`, `LoginPage`, `HomePage`, `MainGameDashboard`, `ProfileDashboard`, `PlayGame`, `GameGUI`, `ChatboxGUI`, `ClientUI` (interface).
- **Client / network:** `ClientCore`, `ClientReceiver`, `NETWORK` stub, `clientGame` (interface).
- **Server / platform:** `ServerCore`, `PlatformCore`, `GameLobby`, `matchmaking`, `Chat`, `Database` stub, `GameSessionManager`, `EventBus`, `GameStateUpdate` (event type), `MatchMaking`, `Leaderboard`, `SessionManager`.
- **Game layer (diagrams):** `GameEngine` interface with `createInitialState` / `validateMove` / `applyMove` / `evaluateEnd`; separate **service** `Game`; `PlayerAction`, `RuleValidator`.
- **02.03 diagram:** Example game **Checkers** (`CheckersGame`).

---

## 4. As-built map (what exists in code)

| Layer | Primary types / packages |
|---|---|
| Entry | `ca.ucalgary.seng300.MainApp` (JavaFX). |
| UI | `ui.*` controllers (e.g. `LoginController`, `GameDashboardController`, `hostLobbyController`, `TicTacToeController`, `Connect4Controller`, `CentralController`, `WindowManager`). |
| Client | `platform.client.ClientCore` (socket I/O, `PlayerAction` serialization, callbacks into UI). |
| Server | `platform.server.PlatformCore` (lobbies, users, routing, matchmaking service); `platform.server.ServerLobby`, `platform.server.platformChannel`. |
| Sessions / games | `platform.session.TicTacToeSession`, `Connect4Session`; `game.core.Game`, `TTTGame`, `C4Game`, `*GameState`, `*RuleValidator`, `*PlayerAction`. |

| Layer | Primary types / packages |
|---|---|
| Services | `platform.services.MatchMakingService`, `LeaderBoardService`. |
| Persistence | `platform.store.*` (`UserStore`, `GameHistoryStore`, … ), `datafiles/` JSON. |
| Stubs | `platform.stubs.Network`, `CredentialsDatabase`; legacy `model.Player` / `model.Rank` may remain for early stub tests — canonical platform user is `platform.domain.User` and game tokens are `game.core.Player`. |

**Move pipeline (simplified):** User input in JavaFX → `ClientCore.submitMove` → server handles `PlayerAction` → game `handleAction` / `RuleValidator` → updated `GameState` → client receives update → `refreshFromGameStateUpdate` on controllers.

---

## 5. Design changes — exact delta (planned → actual)

| # | Area | Planned / documented | As implemented | Rationale |
|---|---|---|---|---|
| 1 | **Game rules API** | `GameEngine` with `createInitialState`, `validateMove`, `applyMove`, `evaluateEnd` (`docs/02 structure/02.03 – Game and Game Engine.md`). | `Game` interface with `startGame`, `handleAction`, `getGameState`, `isGameOver`; validation via `RuleValidator`; state via `GameState` subclasses. | Team standardized on one **`Game`** abstraction plus explicit validators and actions; matches `Full Architecture.md` §4.3 table (`handleAction` / `getGameState`) while differing from the PlantUML `GameEngine` snippet. |
| 2 | **Sample game in class diagram** | Checkers package in 02.03 PlantUML. | **Tic-Tac-Toe** (`TTTGame`, `TicTacToeSession`) and **Connect Four** (`C4Game`, `Connect4Session`) only. | Course requires **two** lightweight games; Checkers was illustrative only. |
| 3 | **Server class split** | Distinct **`ServerCore`** routing all services (`Full Architecture.md` §4.2). | **`PlatformCore`** is the main server entry (`platform.server.PlatformCore`); no separate `ServerCore.java`. | Consolidation into one orchestrator; `ServerCoreTest` in `src/test` exercises **`PlatformCore`** — naming lag vs. implementation. |
| 4 | **GameSessionManager / EventBus** | `GameSessionManager` owns move pipeline; `EventBus` publishes `GameStateUpdate` (`docs/02 structure/02.01`, sequences). | **No** classes named `GameSessionManager` or `EventBus` in `src/main/java`. Session lifecycle and moves are handled inside **`PlatformCore`** and **session/game** types; UI updates use method calls (e.g. `refreshFromGameStateUpdate`) rather than a shared event DTO class. | Simpler integration for the prototype; event-bus pattern described in docs was **not** instantiated as separate types. |
| 5 | **GameStateUpdate artifact** | Dedicated event type with fields (`sessionId`, `recipients`, `status`, … ). | **Concept** preserved as refreshed `GameState` / controller methods; **not** a single `GameStateUpdate` Java class in core. | Same user-visible effect with less boilerplate for two games. |

| # | Area | Planned / documented | As implemented | Rationale |
|---|---|---|---|---|
| 6 | **UI class names** | `WelcomePage`, `MainGameDashboard`, `PlayGame`, etc. | JavaFX **FXML + controllers** (`*Controller`), `MainApp`, `CentralController`, `GameGUI` / game controllers. | Framework idiom: pages map to FXML + controller classes, not one class per paper screen name. |
| 7 | **ClientUI interface** | All UI behind `ClientUI` with `send`/`receive`. | Concrete controllers + `CentralController`/`ClientCore` coupling without a single `ClientUI` interface. | Pragmatic JavaFX structure; interface boundary not used. |
| 8 | **ClientReceiver** | Separate receiver service (`Full Architecture.md`). | Receive loop / dispatch **inside** `ClientCore` (and related wiring). | Merged for threading and I/O co-location. |
| 9 | **Persistence** | Generic **`Database`** stub (`storeData`/`retrieveData`). | **Typed stores** (`UserStore`, `GameHistoryStore`, …) + JSON files + Gson; **`CredentialsDatabase`** still models auth DB boundary. | Clearer domain boundaries and testability; still swappable for a real DB at store interfaces. |
| 10 | **Network stub** | `NETWORK` / TCP stub. | `Network.createConnectedPair()` for tests; **real `Socket`** used in `ClientCore` for runtime (`InetSocketAddress`). | Demonstrate real IPC locally; stubs remain for unit tests. |
| 11 | **Leaderboard / matchmaking names** | `Leaderboard`, `MatchMaking` classes per early diagrams. | `LeaderBoardService`, `MatchMakingService`. | Service suffix + concrete APIs aligned with stores/stats. |
| 12 | **Domain model duplication** | Early QA stub doc (`docs/stubs/README.md`) references `ca.ucalgary.seng300.model.Player`. | **`platform.domain.User`** for accounts; **`game.core.Player`** for X/O or seat; optional legacy **`model.Player`**. | Evolution from stub to DDD-style platform domain; game “player” kept distinct from user account. |

## 6. Testing evidence

Run **`./mvnw test`** (Windows: `mvnw.cmd test`). Suites cover **stores** and persistence, **platform** (`PlatformCore` / server behaviors), **stubs** (`Network`, `CredentialsDatabase`), and **game rules** (e.g. TTT/Connect4 validators and state). See `src/test/java` for layout.

## 7. Known limitations / future work

- **Production DB / hosting:** File-backed stores and local sockets — replace with real persistence and deployment behind the same store/stub seams.
- **Documentation debt:** Sequence diagrams still name `GameSessionManager` / `EventBus`; code path uses `PlatformCore` + direct updates — treat diagrams as **intent**, summary as **as-built**.
- **README:** Minor bugs have been noted in commits (e.g. lobby edge cases); see Git history for outstanding issues.

## 8. Appendix — checklist traceability

- **Baseline sources:** `docs/Iteration 1 Submission/Full Architecture.md`, `docs/02 structure/02.01`, `02.03`, `02.04`, `docs/stubs/README.md`.
- **As-built sources:** `src/main/java/ca/ucalgary/seng300/platform/server/PlatformCore.java`, `platform/client/ClientCore.java`, `platform/session/*`, `game/core/**`, `platform/store/**`, `platform/stubs/**`, `ui/**`.